Design For Test

BY: Eng Ali Eltemsah

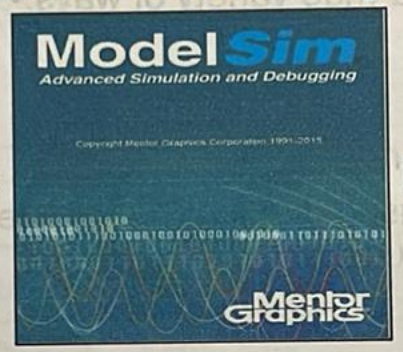
CONTENTS

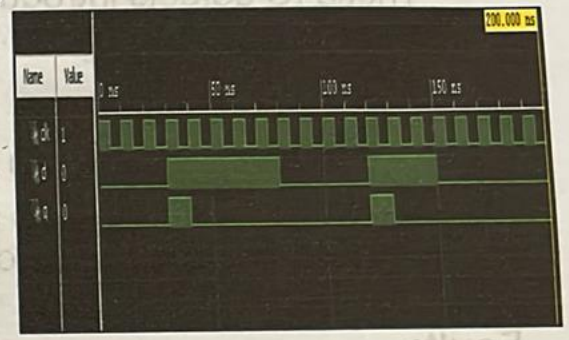
Manufacturing Test

- **There are two basic forms of test:-**

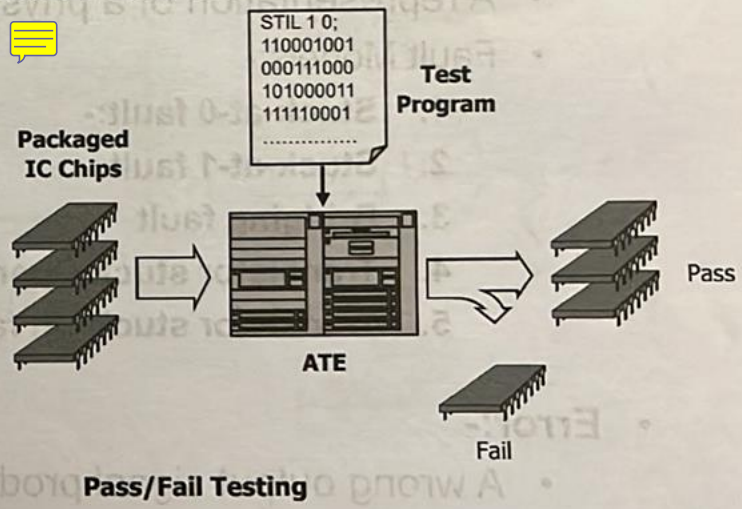
1. **Functional test: Does this chip design produce the correct results?**
   - Functional test seeks logical correctness design through simulation tool.

```
module mux  (
input   wire                    IN_0,
input   wire                    IN_1,
input   wire                    SEL,
output  wire                    mux_out

);

assign mux_out = SEL ? IN_1 : IN_0 ;


endmodule
```



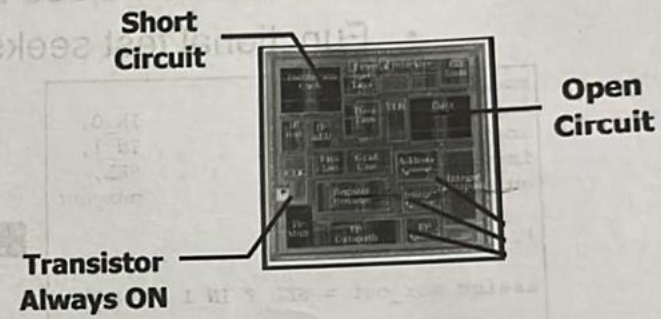


2. **Manufacturing test: Does this particular die work? Can I sell it?**
   - Manufacturing test seeks physical-defects-free ICs.
   - Packaged IC chips are tested using automated test equipment (ATE) through a test program. If a unit fails any test in the program, it is discarded. Only units that pass every test in the program are ever shipped to the end user.
   - Test program or test patterns are generated using automatic test-pattern generation (ATPG) tools.
   - The ATPG tools use the inserted DFT logic to generate the test patterns.



Manufacturing Test

- **Physical defects:-**
  - A on-chip flaw introduced during fabrication(semiconductor processing) or packaging of an individual IC that makes the device **malfunction**.
  - These IC defects introduced in a wide variety of ways:-
    1. Open circuits
    2. Short circuits.
    3. Bridging between metal lines.
  - Most of physical defects are caused due to dust particle or due to fabrication process issues.



- **Fault:-**
  - A representation of a physical defect at the abstracted function level in a circuit.
  - Fault Models:-
    1. **Stuck-at-0 fault:-**
    2. **Stuck-at-1 fault**
    3. **Bridging fault**
    4. **Transistor stuck Open fault**
    5. **Transistor stuck On fault**

- **Error:-**
  - A wrong output signal produced by a defective circuit which causes devices to malfunction

Fault Models

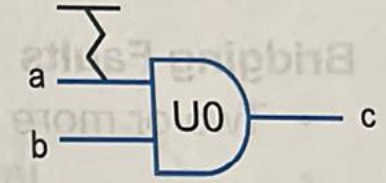
- **Stuck-At-Fault:**
  - Circuit Node has a constant value. It is categorized to:-
    - Stuck-At-0 : Node is shorted to GND
    - Stuck-At-1 : Node is shorted to VDD

a	b	c	c(a/0)	c(a/1)	c(b/0)	c(b/1)	c(c/0)	c(c/1)
0	0	0	0	0	0	0	0	1
0	1	0	0	1	0	0	0	1
1	0	0	0	0	0	1	0	1
1	1	1	0	1	0	1	0	1

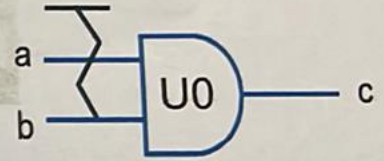
$T_{a/0} = \{11\}; T_{a/1} = \{01\}; T_{b/0} = \{11\}; T_{b/1} = \{10\}$

$T_{c/0} = \{11\}; T_{c/1} = \{00\}$ or $\{01\}$ or $\{10\}$

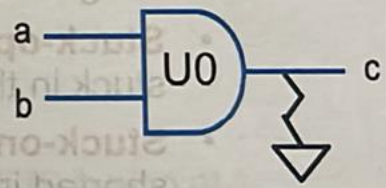
$T = \{01, 10, 11, 00\}$



U0/a
Stuck-At-1



U0/b
Stuck-At-1

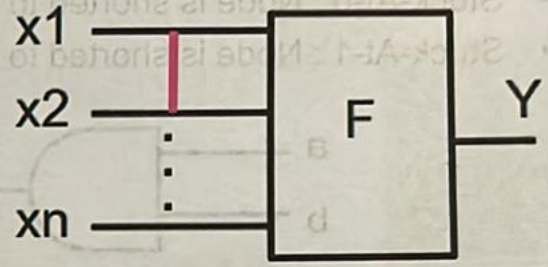


U0/c
Stuck-At-0

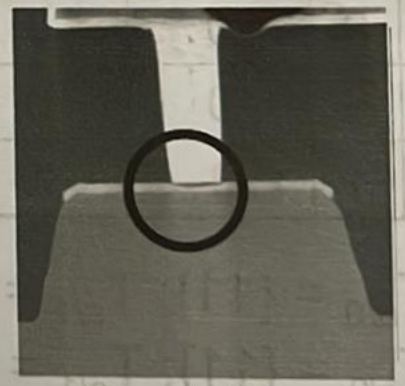
Fault Models

- **Bridging Faults**
  - Two or more normally distinct points (lines) are shorted together

Metal 5 film particle
(bridging defect)



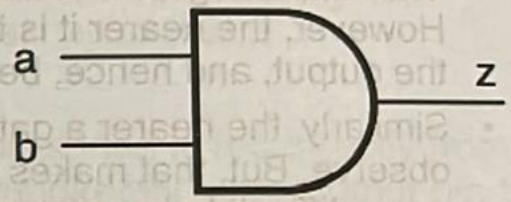
- **Transistor Faults**
  - MOS transistor is considered an ideal switch.
  - It is categorized to two types of faults.
    - **Stuck-open fault:** a single transistor is permanently stuck in the open state the open state
    - **Stuck-on fault:** a single transistor is permanently shorted irrespective of its gate voltage



Open defect

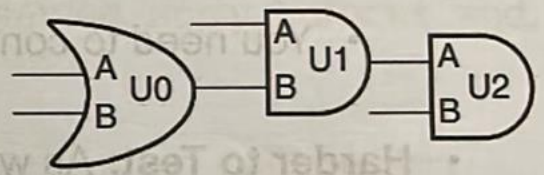
Controllability and Observability

- Testing of any logic gate against stuck-at situations. You need an ability to :-
  - Control any of the inputs (a, b) to 0
  - Control any of the inputs (a, b) to 1
  - Observe the value at the output (c) of the logic gate



- To have a testable design. You have to met two conditions:-
  - All the gates inputs are controllable.
  - The gate output is observable.

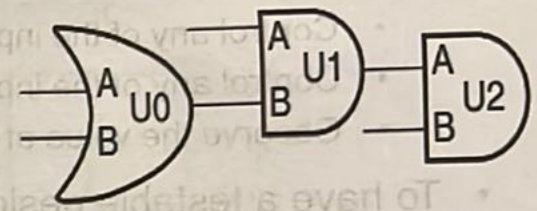
- Controllability and Observability Conflict:-



  A. In order to test **U1** (AND gate) for manufacturing defects So
     1. You need to control U1/B to 0 by control both inputs of U0 to 0
     2. You need to observe the value of U1 by control U2/B to a 1 – so that U1's output can pass through U2.
  B. In order to test **U0** (OR gate) for manufacturing defects So
     1. It is very easy to control U0's input pins.
     2. But in order to observe U0's output, you have to ensure U1/A is at 1 and ~~U2/B is also at 1~~
  C. In order to test **U2** (AND gate) for manufacturing defects So
     1. It is very easy to observe U2's output pins.
     2. But, controlling its inputs might need controlling values at U1/A, U0/A and U0/B

Controllability and Observability

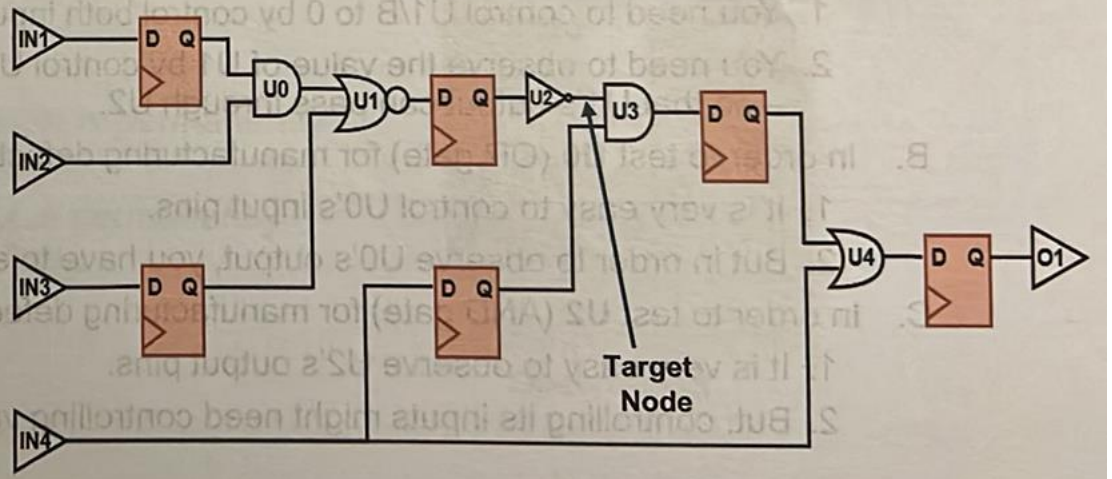
- Controllability and Observability Conflict:-
  - The nearer a gate is towards the input, the easier it is to control. However, the nearer it is towards input, the farther it becomes from the output, and hence, becomes that much more difficult to observe.
  - Similarly, the nearer a gate is towards the output, the easier to observe. But, that makes it farther from the input, making it more difficult to be controlled.



- In a real design, there are millions of gates. so if you need to test the target node.
  - You need to control both U0, U1 and U2 inputs to control U3 input
  - You need to control U3 other input and U4 inputs to observe U3 output

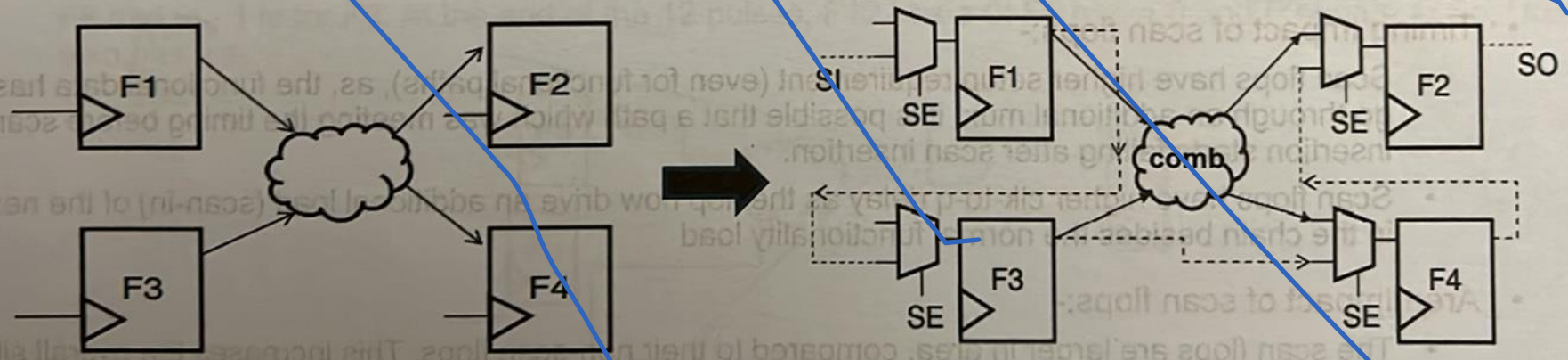
- **Harder to Test:** As we need to control all the internal gates inputs through the finite primary Input ports

- The solution is that we need many **Control points** & **Observe Points**



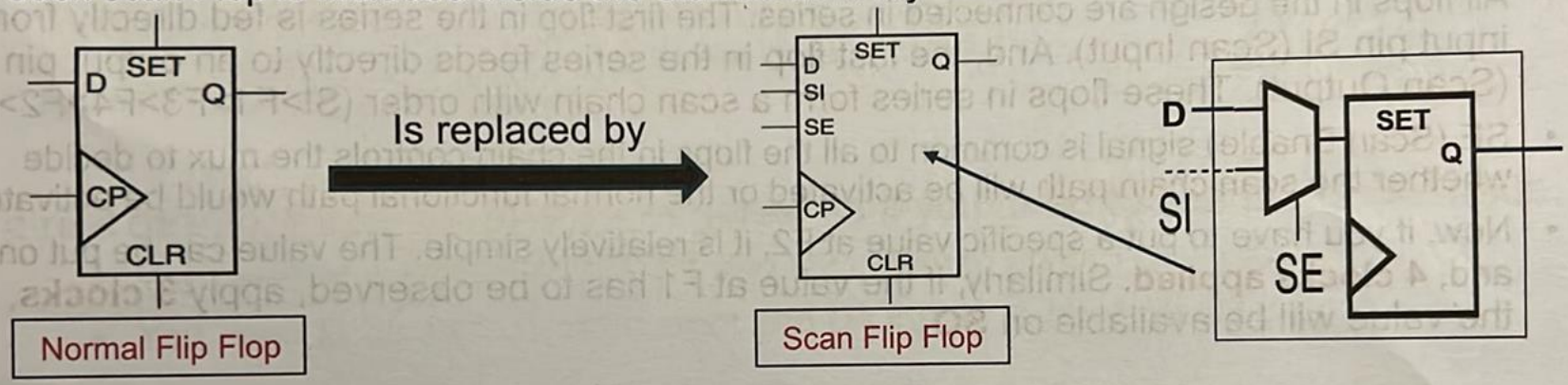
Scan Chains

- Using **Scan Chain** for Controllability and Observability.
  - Since, it is difficult to put in so many extra inputs and outputs for better observability and controllability, thus, as an alternate mechanism which is known as "scan chains".
  - All the Flip Flop is replaced by (Mux + Normal D Flip Flop).
  - All flops in the design are connected in series. The first flop in the series is fed directly from an input pin SI (Scan Input). And, the last flop in the series feeds directly to an output pin SO (Scan Output). These flops in series form a scan chain with order (SI>F1>F3>F4>F2>SO)
  - SE (Scan Enable) signal is common to all the flops in the chain controls the mux to decide whether the scan chain path will be activated or the normal functional path would be activated.
  - Now, if you have to put a specific value at F2, it is relatively simple. The value can be put onto **SI**, and, **4 clocks applied**. Similarly, if the value at F1 has to be observed, apply **3 clocks**, and, the value will be available on **SO**



Scan Chains

- **Scan Flip Flop.**
  - A scan flip-flop is a D flip-flop with a 2x1 multiplexer added at its input D, one input of the MUX acting as the functional input D when SE=0, and the other input serving as the Scan-In (SI) input when SE=1; SE is the MUX control.
  - Such scan Flop is included inside the standard library cell.



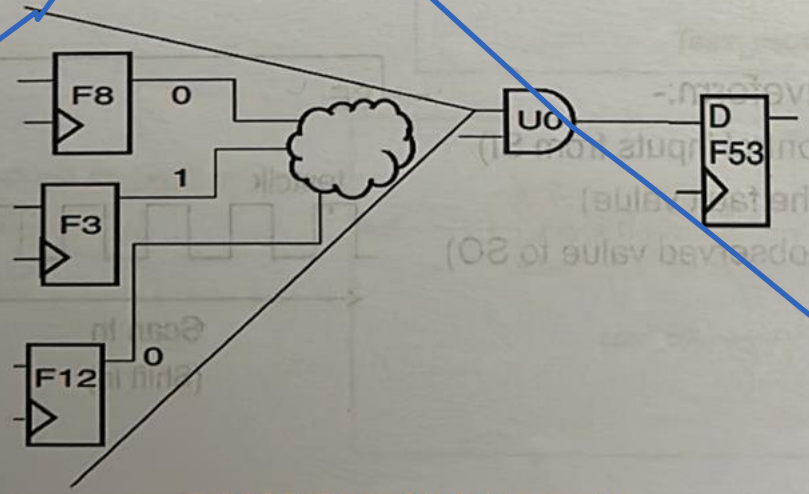
  - Timing Impact of scan flops:-
    - Scan flops have higher setup requirement (even for functional paths), as, the functional data has to go through an additional mux. it is possible that a path which was meeting the timing before scan insertion starts failing after scan insertion.
    - Scan flops have higher clk-to-q delay as the flop now drive an additional load (scan-in) of the next flop in the chain besides the normal functionality load
  - Area Impact of scan flops:-
    - The scan flops are larger in area, compared to their non-scan flops. This increases the overall silicon area of the design. Though, silicon real-estate is very costly, however, this increase in area is a relatively much smaller price to pay, compared to the huge increase in controllability and observability

Design For Testing

Mechanics of Scan Chain: The concept of scan testing depends fully on a series of shifts and captures.

1. ShiftIn (Scan In)

- Consider a specific cell's specific pin is desired to be taken to a given value. You could determine the fanin cone of this pin to the nearest flops. This cone might have N flops. You should be able to determine the values on each of these flops that will result in the desired value at the point of interest. You can count the position of each of these flops in the chain. And, you can put the values into those flops by simply pushing in the values in the right sequence at the SI pin, and, giving the clocks, till all the values reach the desired flops. Since you want the values to be moved across the flops along the scan chain, so, SE is kept asserted.
- Say, you want U0/A to get a value of 1. You traverse the fanin cone of the pin and get three flops. F in the schematic denotes the sequence number of the flop in the scan-chain. The value shown at the flop output shows the value that the flop should have, so that U0/A gets a 1. Now, you need to feed the pattern "0xxx0xxxx1xx" into SI and apply 12 pulses of clock. The first 0 is for F12; the next 0 is for F8 and the 1 is for F3. At the end of the 12 pulses, F12 has a 0; F8 has a 0 and F3 has a 1. So, U0/A also has a 1.

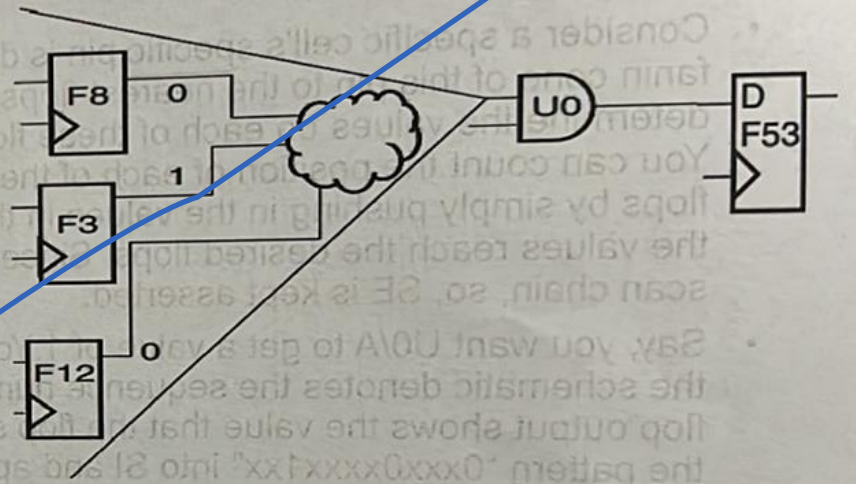


Design For Testing

Mechanics of Scan Chain: The concept of scan testing depends fully on a series of shifts and captures.

2. Capture

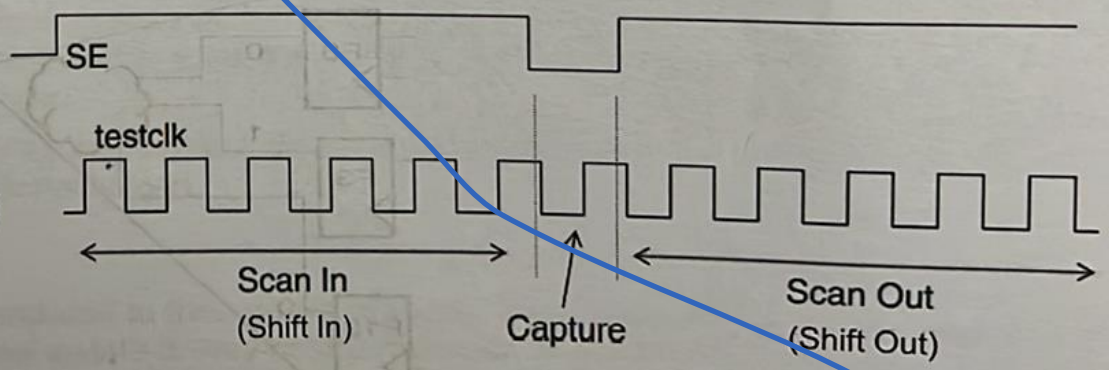
- At this instant, SE is de-asserted, and, another clock is applied. This will cause the U0 output to be sampled by the flop in the immediate fanout. This will be flop F53.



3. ShiftOut (Scan Out)

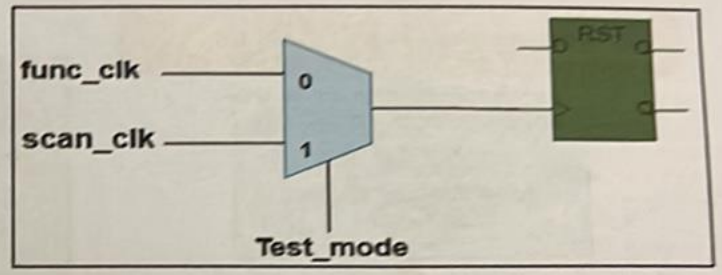
- Now, SE is asserted again, and, the value stored in the Capture Flop is moved along the scan chain till it comes out at SO.

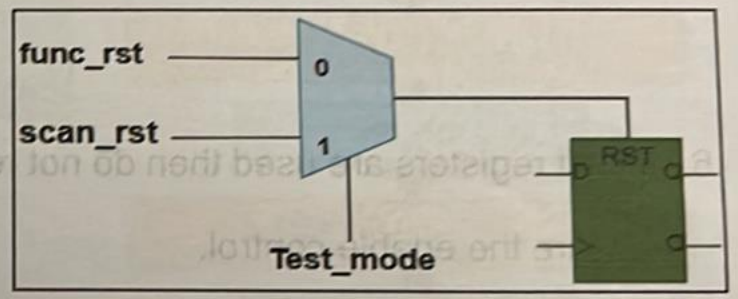
- As shown in the waveform:-
  1. SE=1 (Shift In control inputs from SI)
  2. SE=0 (Capture the fault value)
  3. SE=1 (Shift Out observed value to SO)

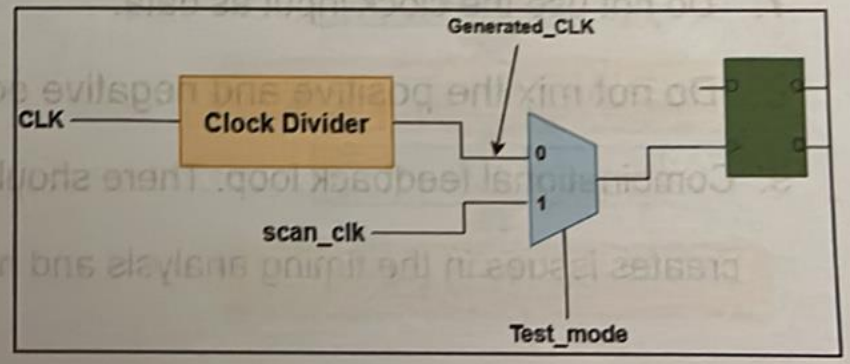


Design For Testing

- **Guidelines:-**
  1. Flops need to have all their control inputs being directly controllable by ATPG tool. These control inputs include data, clocks and asynchronous set/reset must be controlled in the test mode.
  2. All the primary input clocks must be muxed with scan clock in test mode.
  3. All the primary input and internally generated asynchronous resets/sets must be muxed with scan reset/set in test mode
  4. All Generated clocks must be muxed with scan clock in the test mode.

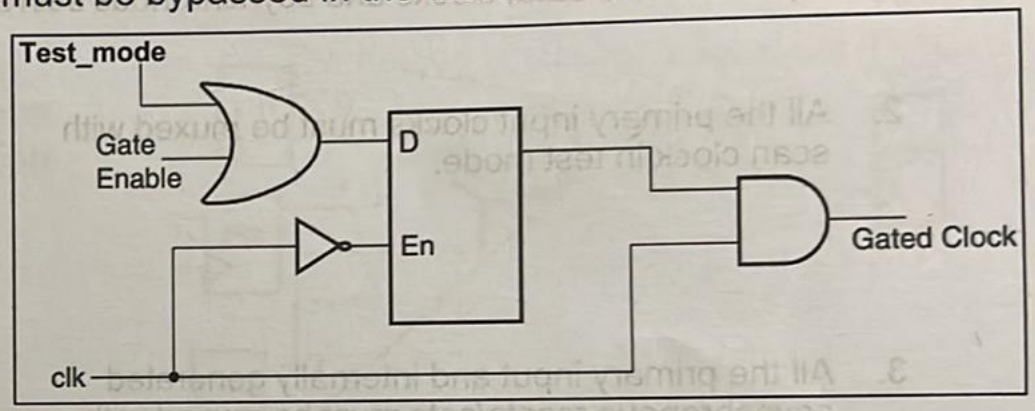






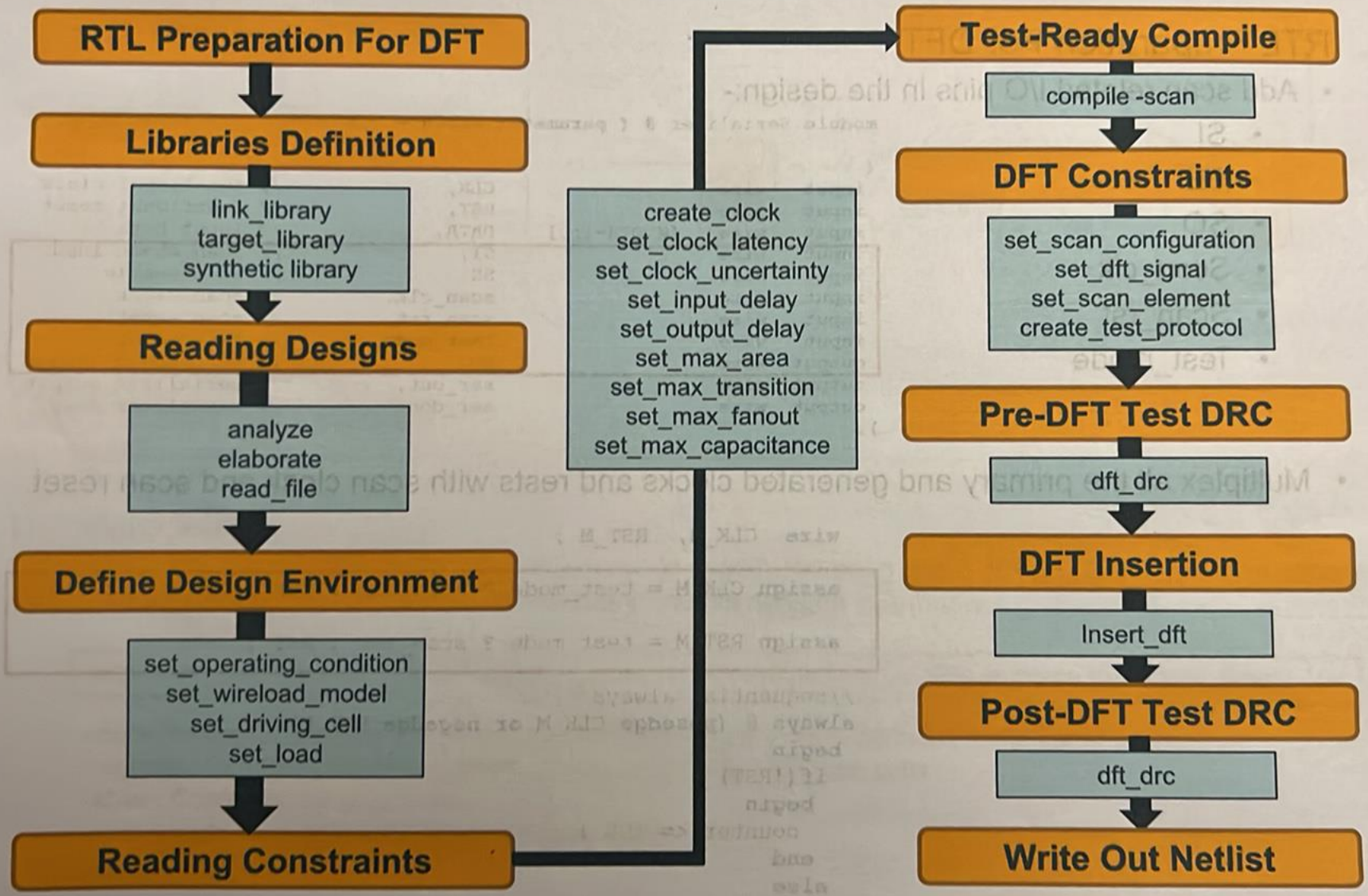
Design For Testing

- **Guidelines:-**
  5. Gated clocks are not controllable and must be bypassed in the test mode.



  6. If shift registers are used then do not replace them by using scan enabled flip-flops but only ensure the enable control.
  7. Do not use the clock input as data.
  8. Do not mix the positive and negative edge triggered flip-flops
  9. Combinational feedback loop: There should not be any combinational loop in the design as it creates issues in the timing analysis and hence it is avoid the manufacturing testability

DFT Flow



DFT Compiler Flow

1. RTL Preparation For DFT.
   - Add scan related I/O pins in the design:-
     - SI
     - SE
     - SO
     - Scan_clk
     - Scan_rst
     - Test_mode

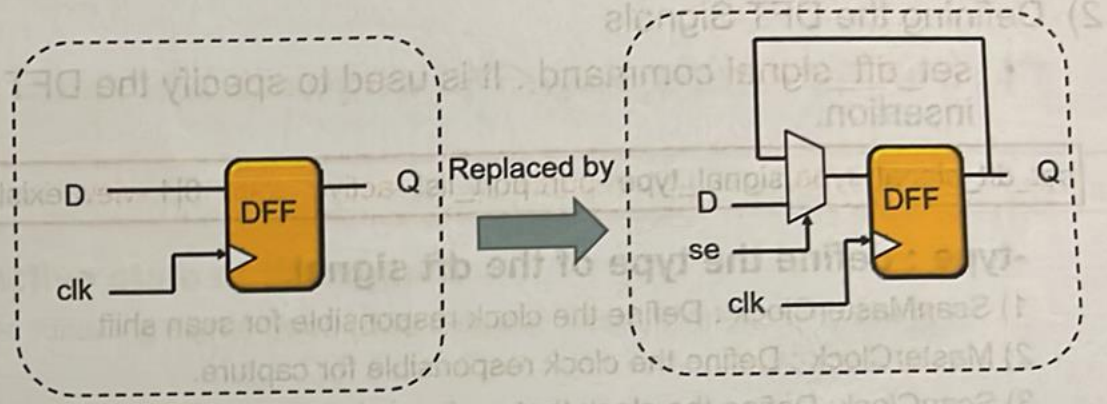
```
module Serializer # ( parameter WIDTH = 8 )

(
 input   wire                    CLK,          // functional clock
 input   wire                    RST,          // functional reset
 input   wire   [WIDTH-1:0]      DATA,         // Input Data
 input   wire                    SI,           // scan chain input
 input   wire                    SE,           // scan enable
 input   wire                    scan_clk,     // scan clock
 input   wire                    scan_rst,     // scan reset
 input   wire                    test_mode,    // test mode
 output  wire                    SO,           // scan chain output
 output  wire                    ser_out,      // serializer output
 output  wire                    ser_done      // serializer done
);
```

   - Multiplex all the primary and generated clocks and rests with scan clock and scan reset.

```
wire  CLK_M,  RST_M ;

assign CLK_M = test_mode ? scan_clk : CLK ;

assign RST_M = test_mode ? scan_rst : RST ;

//sequential always
always @ (posedge CLK_M or negedge RST_M)
 begin
  if(!RST)
   begin
    counter <= 'b0 ;
   end
  else
```

Test-Ready Compile

- compile -scan
- compile_ultra -scan



DFT Constraints

1) Configuring scan chain

set_scan_configuration command : it enables you to specify the scan chain design. This command controls almost all aspects of how the insert_dft command makes designs scannable.

```
set_scan_configuration -clock_mixing no_mix  -style multiplexed_flip_flop -replace true -max_length 100
```

-clock_mixing : Configure the mixing of cells from different clock domains in the same scan chain.
-replace : Configure the replacement of sequential cells with scan cells
-style : Configure the scan style
-max_length : Configure the length of scan chains

DFT Constraints

2) Defining the DFT Signals

- set_dft_signal command : It is used to specify the DFT signal ports for DRC and DFT insertion.

```
set_dft_signal -type signal_type -port port_list -active_state  0|1 -view existing_dft | spec -usage scan | clock_gating
```

-type : define the type of the dft signal.

1) ScanMasterClock : Define the clock responsible for scan shift.
2) MasterClock : Define the clock responsible for capture.
3) ScanClock: Define the clock that performs both scan shift and capture in a MUX-D scan design.
   (This type is equivalent to defining the clock as both the MasterClock and ScanMasterClock)
4) ScanEnable : Define the shift enable signal of a MUX-D scan design.
5) ScanDataIn : Define the scan input ports of the design.
6) ScanDataOut : Define the scan output ports of the design.
7) Reset : Define a asynchronous set or reset of the design.
8) Constant : Define a continuously applied value to a port.
9) TestMode : Define a continuously applied value to a test mode port that may have different values between test modes.

-view : define the type of the dft signal

1) existing_dft : The existing DFT view describes that the signal network is existed (signal is routed).
2) spec: It indicates that the signal network or connection does not yet exist, and the insert_dft command has to route these signals to the scannable cells.

DFT Constraints

2) Defining the DFT Signals

- set_dft_signal command : It is used to specify the DFT signal ports for DRC and DFT insertion.

```
set_dft_signal -type signal_type -port port_list -active_state  0|1 -view existing_dft | spec -usage scan | clock_gating
```

-active_state : define the active state of the signal.

-- Specifies the active states for the following signal types (ScanEnable, Reset, Constant, TestMode)

-usage : define the usage of the dft signal

1) clock_gating : it specifies that the signal is to be connected to the test pin of identified clock gating cells during insert_dft. This usage is valid only for -type ScanEnable and -type TestMode.

2) scan : it specifies that the signal is to be connected to the enable of scan elements during insert_dft. This usage is valid only for -type ScanEnable.

- Examples:-

```
set_dft_signal -port [get_ports scan_clk] -type ScanClock -view existing_dft -timing {30 60}
set_dft_signal -port [get_ports scan_rst] -type Reset -view existing_dft -active_state 0
set_dft_signal -port [get_ports test_mode]   -type Constant   -view existing_dft -active_state 1
set_dft_signal -port [get_ports test_mode]   -type TestMode  -view spec  -active_state 1
set_dft_signal -port [get_ports SE] -type ScanEnable -view spec  -active_state 1  -usage scan
set_dft_signal -port [get_ports clk_gate_byp] -type ScanEnable -view spec -active_state 1  -usage clock_gating
set_dft_signal -port [get_ports SI]  -type ScanDataIn  -view spec
set_dft_signal -port [get_ports SO] -type ScanDataOut -view spec
```

DFT Constraints

3) Creating Test Protocol

- create_test_protocol command creates a test protocol for the current design based on user specifications issued using **set_dft_signal** commands and check whether the user-specified values are consistent with each other. If they are not, it issues an error and does not generate a protocol. create_test_protocol command should be executed before running the dft_drc command because design rule checking requires a test protocol.

```
create_test_protocol
```

```
In mode: all_dft...

Information: Starting test protocol creation. (TEST-219)
  ...reading user specified clock signals...
Information: Identified system/test clock port scan_clk (30.0,60.0).
  ...reading user specified asynchronous signals...
Information: Identified active low asynchronous control port scan_rst
```

4) Performing Pre-DFT Test DRC

- dft_drc command analyzes your unrouted(unstitched) scan design or analyze your design before using insert_dft command, based on a set of constraints applicable to your selected scan style, If test DRC reports no violations, you can insert DFT structures into your design using insert_dft Command and if reports a set of violations. Based on the violations, you make changes to your design to prepare it for DFT insertion.

```
dft_drc -verbose
```

```
dft_drc -verbose
In mode: all_dft...
  Pre-DFT DRC enabled

Information: Starting test design rule checking.
  Loading test protocol
  ...basic checks...
  ...basic sequential cell checks...
        ...checking for scan equivalents...
  ...checking vector rules...
  ...checking pre-dft rules...

  DRC Report

  Total violations: 0

--------------------------------------------------------------------------------

Test Design rule checking did not find violations

--------------------------------------------------------------------------------
  Sequential Cell Report

  0 out of 71 sequential cells have violations

--------------------------------------------------------------------------------

SEQUENTIAL CELLS WITHOUT VIOLATIONS
      *  71 cells are valid scan cells
```

DFT Constraints

5) Previewing Scan Insertion

- preview_dft command runs the same scan architecture algorithms as the insert_dft command, except that it reports the scan architecture to be implemented instead of actually implementing it.

```
preview_dft -show scan_summary
```

```
****************************************
Preview_dft report
For    : 'Insert_dft' command
Design : ALU_TOP
Version: K-2015.06
Date   : Fri Apr 15 06:42:39 2022
****************************************

Number of chains: 1
Scan methodology: full_scan
Scan style: multiplexed_flip_flop
Clock domain: no_mix

Chain      Scan Ports (si --> so)        # of Cells   Inst/Chain                  Clock (port, time, edge)
-----      ----------------------        ----------   ----------                  ------------------------
S 1        SI -->  SO                            71   U0_ARITHMETIC_UNIT/Arith_Flag_reg
                                                      (scan_clk, 30.0, rising)
```

6) DFT Insertion

- insert_dft command building the scan chains by doing the following:-
  - Remapping nonscan sequential cells to library scan equivalent cells
  - Converting the scan elements that resulted from a test-ready compile or a previous scan insertion back to nonscan elements if test DRC violations prevent their inclusion in a scan chain and issued this command
  - Routing global test signals to the specified scan flip flops. These test signals include clocks, rests and enable signals, scan chain inputs and scan chain outputs.
  - Stitching scan chain by connecting scan flops together

```
insert_dft
```

```
  Information: Using test design rule information from previous dft_drc run.
  Architecting Scan Chains
  Routing Scan Chains
  Routing Global Signals
  Mapping New Logic
Resetting current test mode
```

DFT Constraints

7) Post-DFT Optimization

- Post-DFT optimization is gate-level optimization performed after inserting and mapping the newly inserted DFT logic.

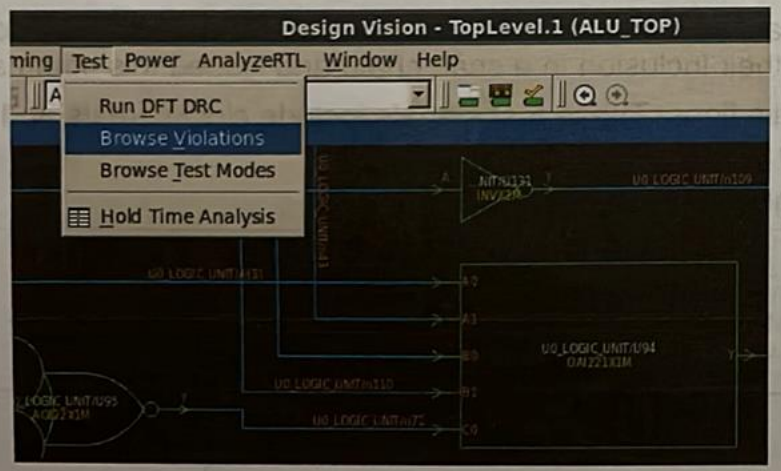
```
compile -scan -incremental
```

8) Performing Post-DFT Test DRC

- dft_drc command analyzes your routed and stitched scan design after dft insertion
- - coverage_estimate : it generates test coverage statistics for the current design.

```
dft_drc -verbose -coverage_estimate
```

- DRCs Violation debugging through DC GUI




```
dft_drc -verbose -coverage_estimate
In mode: Internal_scan...
  Design has scan chains in this mode
  Design is scan routed
  Post-DFT DRC enabled
-------------------------------------------------
 DRC Report

 Total violations: 0
-------------------------------------------------
   Uncollapsed Stuck Fault Summary Report
-------------------------------------------------
fault class                     code    #faults
-------------------------------------------------
Detected                         DT       12066
Possibly detected                PT           0
Undetectable                     UD          13
ATPG untestable                  AU          11
Not detected                     ND          24
-------------------------------------------------
total faults                                12114
test coverage                               99.71%
-------------------------------------------------
```